

Дизайн и анализ на алгоритми упражнение 14

Станислав Игов (Стъката)

2025/2026

NP-Пълнота и Класически Редукции. Решими и нерешими задачи. Класове P и NP. КНФ. Полиномиални редукции. SAT, 3-SAT, Vertex Cover, Partition.

Част 1: Какво е изчислима задача?

Преди да говорим за P и NP, нека стъпим на по-основен въпрос: **кои задачи изобщо могат да бъдат решени от компютър?**

Решими и нерешими задачи

Изчислимостта е фундаментална граница — тя ни казва дали дадена задача *принципно може* да бъде решена от машина, независимо колко бърза е тя.

Решими задачи (Decidable). Една задача е **решима**, ако съществува алгоритъм (машина на Тюринг), който **винаги спира** (за всеки вход) и дава **правилен отговор** (YES или NO).

Примери за решими задачи:

Задача	Защо е решима?
“Дадено число n — просто ли е?”	Алгоритъмът пробва всички делители до $\sqrt{n}$ и спира
“Два сортирани масива равни ли са?”	Сравнява елемент по елемент и спира
“Регулярно изражение — приема ли дума w ?”	Автоматите за регулярни езици винаги спират

Нерешими задачи (Undecidable). Една задача е **нерешима**, ако *не съществува никакъв алгоритъм*, който да я реши за всеки вход. Не е въпрос за бавност — такъв алгоритъм просто не може да съществува.

Най-известният пример: Проблемът за спирането (Halting Problem).

Задача: Дадени са програма P и вход x . Ще спре ли P при изпълнение с вход x ?

Алън Тюринг доказва през 1936 г., че **никой алгоритъм не може да реши тази задача за всички възможни входове**. Доказателството е чрез диагонален аргумент — ако допуснем, че такъв алгоритъм H съществува, можем да конструираме програма, която го противоречи на себе си.

Още примери за нерешими задачи:

- “Дадена програма — ще изпечата ли Hello World за поне един вход?” (*Теорема на Райс*)
- “Дадена контекстно-свободна граматика — съвпада ли нейният език с Σ^* ?”
- “Дадено уравнение на Диофант (с цели числа) — има ли решение?” (*10-ти проблем на Хилберт*)

Част 2: Класове на сложност

Задачи за решение (Decision Problems)

Всички задачи в теорията на сложността са формулирани като **задачи за решение** — такива с отговор YES или NO. Много задачи за оптимизация могат да се превърнат в задачи за решение:

Оптимизационна задача	Задача за решение
“Намери най-краткия маршрут от A до B ”	“Съществува ли маршрут с дължина $\leq k$?”
“Намери максимално независимо множество”	“Съществува ли независимо мн-во с размер $\geq k$?”
“Намери минимално върхово покритие”	“Съществува ли върхово покритие с размер $\leq k$?”

Клас P — “Бързо решими”

Дефиниция: Класът **P** съдържа всички задачи за решение, които могат да се решат за **полиномиално** време от детерминирана машина на Тюринг.

Полиномиално = $O(n^k)$ за някоя константа k , където n е размерът на входа.

Примери за задачи в P:

Задача	Алгоритъм	Сложност
Сортиране	Merge Sort	$O(n \log n)$
Намиране на най-кратък път	Dijkstra	$O((V + E) \log V)$
Проверка за прости числа	AKS тест	$O(n^{12})$ (полиномиален!)
Намиране на максимален поток	Ford–Fulkerson с BFS	$O(VE^2)$
Минимално покриващо дърво	Kruskal	$O(E \log V)$
2-оцветяване на граф	BFS	$O(V + E)$
2-SAT	Kosaraju / Tarjan	$O(V + E)$

Клас NP — “Бързо проверими”

NP идва от **Nondeterministic Polynomial time**, но най-интуитивната дефиниция е:

Клас NP съдържа всички задачи за решение, за които когато отговорът е **YES**, съществува **кратко доказателство (сертификат / свидетел)**, което може да бъде **проверено за полиномиално време**.

Какво е “сертификат”? Сертификатът е “подсказка” или “доказателство” за отговора YES. Той трябва да има **полиномиален размер** спрямо входа и да може да се **провери за полиномиално време**.

Примери за задачи в NP:

- **SAT (Boolean Satisfiability).**

- Задача: Дадена булева формула ϕ — съществува ли присвояване на променливите, което я прави True?
- Сертификат: Конкретното присвояване (напр. $x_1 = T$, $x_2 = F$, $x_3 = T$).
- Проверка: Заместваем стойностите в ϕ и изчисляваме — $O(|\phi|)$ стъпки.

- **Vertex Cover.**

- Задача: Даден граф G и число k — съществува ли множество от върхове S с $|S| \leq k$, такова че всяко ребро е инцидентно с поне един връх в S ?
- Сертификат: Конкретното множество S .
- Проверка: Проверяваме всяко ребро дали поне единият му край е в S — $O(|E|)$ стъпки.

- **Hamiltonian Path.**

- Задача: Даден граф G — съществува ли път, минаващ точно веднъж през всеки връх?
- Сертификат: Конкретната наредба на върховете.
- Проверка: Проверяваме дали всяка двойка последователни върхове е ребро — $O(|V|)$ стъпки.

- **Subset Sum.**

- Задача: Дадено множество от числа A и цел K — съществува ли подмножество, чиято сума е точно K ?
- Сертификат: Конкретното подмножество.

- Проверка: Сумираме елементите — $O(n)$ стъпки.

Ключово наблюдение:

Намирането на решение може да е **МНОГО** трудно. Проверката дали дадено решение е вярно е **ЛЕСНО**.

Аналогия от живота: Решаването на sudoku е трудно. Проверката дали попълнено sudoku е вярно е лесно — просто проверяваш всеки ред, колона и квадрат.

Пример от живота: Разбиване на парола на телефон

Представи си телефон с **4-цифрен PIN** — цифри от 0 до 9.

Намирането на паролата (без подсказка) изисква да пробваш до 10 000 комбинации — трудно.

Проверката дали дадена комбинация е вярна — просто я въвеждаш. Едно опитване = $O(1)$.

Това е точно духът на NP:

- Търсенето на сертификата (паролата) е трудно.
- Проверката на сертификата е тривиално лесна.

Нека паролата е **6 символа** от 94 printable ASCII символа:

	Намиране	Проверка
Груба сила	~ 689 милиарда комбинации	Веднага — въвеждаш и гледаш
Сложност	Експоненциална в дължината	$O(\text{дължина на паролата})$

- **Сертификатът** = самата парола.
- **Верификаторът** = телефонът, който проверява дали съвпада.

Ако $P = NP$, то щом телефонът може бързо да *провери* паролата, би съществувал алгоритъм, който бързо я *намира* — всяка криптография би рухнала.

Релацията $P \subseteq NP$

Важен факт: $P \subseteq NP$.

Всяка задача в P е и в NP. Защо? Ако можем да *решим* задачата за полиномиално време, то можем и да я *проверим* за полиномиално (просто игнорираме сертификата и решаваме от нулата).

+-----+			
	NP		
	+-----+		
		P	
		- Sorting	
		- Shortest Path	
		- Max Flow	
	+-----+		
	- SAT	- Vertex Cover	
	- Hamiltonian Path		
	- Subset Sum	- 3-Coloring	
+-----+			

Великият въпрос: $P = NP$?

Един от **най-важните нерешени проблеми в математиката и компютърните науки** е:

Дали $P = NP$?

Накратко: “Ако можеш бързо да *проверяваш* решения, можеш ли и бързо да ги *намираш*?”

- Ако $P = NP$: Всяка задача, за която можем бързо да проверяваме решения, може и бързо да се реши. Това би революционизирало криптографията (RSA ще се счупи), медицината, логистиката. . .
- Ако $P \neq NP$ (което повечето учени смятат за вярно): Има задачи, за които проверката е лесна, но намирането е принципно трудно.

Защо е важно? Повечето криптографски системи (HTTPS, банкови транзакции) разчитат на $P \neq NP$ — те са сигурни, защото се вярва, че разбиването им е NP-трудно.

Статус: Задачата е включена в **Millennium Prize Problems** на Clay Mathematics Institute. Решението носи **1 000 000** долара награда и е отворена от 1971 г.

За упражнението приемаме $P \neq NP$, т.е. NP-пълните задачи са принципно по-трудни от P задачите.

Част 3: КНФ — Конюнктивна Нормална Форма

Много NP задачи са свързани с булеви формули в КНФ. Нека обясним подробно.

Булеви операции

Операция	Символ	Значение
AND (И)	$\wedge$	True само ако и двата операнда са True
OR (ИЛИ)	$\vee$	True ако поне един операнд е True
NOT (НЕ)	$\neg$	Обръща стойността

Терминология (разглежда се по-подробно в курса по логическото програмиране)

Термин	Определение	Пример
Атом	Най-простото твърдение — без вътрешна структура	p, q, x_1, rain
Литерал	Атом или неговото отрицание	$x_1, \neg x_1, \neg \text{rain}$
Дизюнкт	Крайна дизюнкция (OR) на литерали — <i>клаузата в КНФ</i>	$(x_1 \vee \neg x_2 \vee x_3)$
Конюнкт	Крайна конюнкция (AND) на формули	$(D_1 \wedge D_2 \wedge D_3)$
КНФ формула	Конюнкт от дизюнкти — AND на OR-ове	$D_1 \wedge D_2 \wedge \dots \wedge D_m$

В логическото програмиране (Prolog и сродни) дизюнктите се записват като **правила и факти**. *Клаузата на Хорн* е специален вид дизюнкт с **най-много един положителен литерал** — това е основата на Prolog.

Конюнктивна Нормална Форма (КНФ)

Дефиниция: Формула е в **КНФ**, ако е конюнкт от дизюнкти — тоест AND на OR-ове:

$$\phi = D_1 \wedge D_2 \wedge \dots \wedge D_m$$

където всеки дизюнкт D_j е дизюнкция на литерали:

$$D_j = (L_{j,1} \vee L_{j,2} \vee \dots \vee L_{j,k})$$

и всеки литерал $L_{j,i}$ е атом или отрицание на атом.

Пример с пълна анотация:

$$\phi = \underbrace{(x_1 \vee \neg x_2)}_{\text{дизюнкт } D_1} \wedge \underbrace{(\neg x_1 \vee x_2 \vee x_3)}_{\text{дизюнкт } D_2} \wedge \underbrace{(x_2 \vee \neg x_3)}_{\text{дизюнкт } D_3}$$

- x_1, x_2, x_3 — **атоми**.
- $x_1, \neg x_2, \neg x_1, x_2, x_3, \neg x_3$ — **литерали**.
- $(x_1 \vee \neg x_2), (\neg x_1 \vee x_2 \vee x_3), (x_2 \vee \neg x_3)$ — **дизюнкти**.
- Цялата ϕ — **конюнкт** от дизюнкти = КНФ формула.

Ще бъде ли изпълнима при $x_1 = T, x_2 = T, x_3 = F$?

Дизюнкт	Оценка
$D_1 = (x_1 \vee \neg x_2) = (T \vee F)$	True
$D_2 = (\neg x_1 \vee x_2 \vee x_3) = (F \vee T \vee F)$	True
$D_3 = (x_2 \vee \neg x_3) = (T \vee T)$	True

$\phi = T \wedge T \wedge T = \text{True}$ — формулата е **изпълнима**!

Защо КНФ е важна?

- **Удобна структура:** Лесно се проверява дали дадено присвояване удовлетворява всеки дизюнкт поотделно.
- **SAT е NP-пълна:** Доказва се (теорема на Кук–Левин, 1971) — SAT е първата доказана NP-пълна задача.
- **Основа за редукции:** Повечето NP-пълни задачи се доказват чрез свеждане от SAT или 3-SAT.

k-SAT

k-SAT е специален случай на SAT, при който **всеки дизюнкт съдържа точно k литерала**.

Вариант	Дизюнкти	Сложност
1-SAT	по 1 литерал	В P (тривиален)
2-SAT	по 2 литерала	В P (чрез силно свързани компоненти)
3-SAT	по 3 литерала	NP-пълна
k-SAT ($k \geq 3$)	по k литерала	NP-пълна

Забележително: 2-SAT е в P, но 3-SAT е NP-пълна! Добавянето на само един литерал на дизюнкт преминава граница.

Част 4: NP-Hard и NP-пълни задачи

NP-Hard

Дефиниция: Задача A е **NP-hard**, ако **всяка** задача от NP може да се сведе до A за полиномиално време.

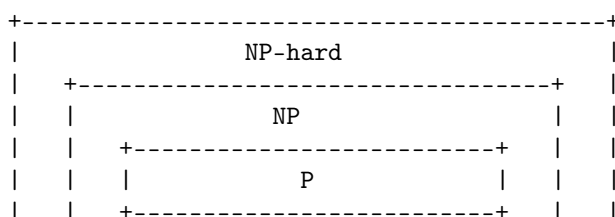
- NP-hard задачите са **поне толкова трудни** като всяка NP задача.
- Те *не трябва* задължително да са в NP!
- Пример за NP-hard задача, която не е в NP: **Halting Problem** (нерешима, но по-трудна от всяка NP задача).

NP-пълни задачи

Дефиниция: Задача A е **NP-пълна**, ако:

1. $A \in \text{NP}$ (може да се провери за полиномиално време).
2. A е NP-hard (всяка NP задача се свежда до A).

$$\text{NP-пълна} = \text{NP} \cap \text{NP-hard}$$



		NP-пълни задачи		
		(в NP пресечено с NP-hard)		
	+-----+			
	Задачи извън NP (нерешими и др.)			
+-----+				

Класически NP-пълни задачи:

- SAT, 3-SAT
- Vertex Cover, Independent Set, Clique
- Hamiltonian Path / Hamiltonian Cycle
- Travelling Salesman (TSP) — решение вариант
- Graph Coloring (k -Coloring за $k \geq 3$)
- Subset Sum, Partition, Knapsack

Част 5: Полиномиална редукция

Интуиция

Полиномиалната редукция е “превод” от един проблем в друг. Ако знаем как да решим задача B , и знаем как да “преведем” задача A в задача B , тогава можем да решим и A чрез B .

Аналогия: Представете си, че знаете как да решавате уравнения от вида $ax = b$. Някой ви дава уравнение $3(x + 2) = 15$. Вие го *преформулирате*: $3x + 6 = 15 \implies 3x = 9$ — сега е в познатата форма $ax = b$. Решавате с $x = 3$.

Това е редукция: *трансформирахте* новата задача в позната.

Формална дефиниция

Дефиниция: Задача A е **полиномиално сводима** до задача B ($A \leq_p B$), ако съществува функция f , изчислима за полиномиално време, такава че:

$$x \in A \iff f(x) \in B$$

- f се нарича **редукция** (reduction / свеждане).
- f се изчислява за **полиномиално** (не експоненциално!) време.
- Ако x е YES-инстанция за A , тогава $f(x)$ е YES-инстанция за B .
- Ако x е NO-инстанция за A , тогава $f(x)$ е NO-инстанция за B .

Защо посоката на редукцията е важна

$A \leq_p B$ означава:

“ A е не по-трудна от B ” / “ B е поне толкова трудна като A ” / “Ако решаваме B , решаваме и A ”

Запомнете: При доказване на NP-пълнота редуцираме **от известна трудна задача към новата задача**:

Известна NP-пълна $\leq_p$ Нова задача

Тоест показваме, че новата задача е *поне толкова трудна* като вече известна NP-пълна задача.

Грешна посока (честа грешка!): Ако покажем Нова $\leq_p$ Известна NP-пълна, значи само сме показали, че новата задача е **не по-трудна** от известната — не доказваме, че новата е NP-пълна!

Схема за доказване на NP-пълнота

За да докажем, че задача X е NP-пълна:

Стъпка 1: Покажете, че $X \in \text{NP}$.

- Опишете сертификата (какво е “доказателството за YES”).
- Опишете алгоритъма за проверка.
- Покажете, че проверката е полиномиална.

Стъпка 2: Покажете, че X е NP-hard.

- Изберете известна NP-пълна задача Y .
- Конструирайте редукция f : трансформирайте вход x за Y в $f(x)$ — вход за X .
- Покажете **коректност**: $x \in Y \iff f(x) \in X$.
 - Посока ($\Rightarrow$): Ако x е YES за Y , тогава $f(x)$ е YES за X .
 - Посока ($\Leftarrow$): Ако $f(x)$ е YES за X , тогава x е YES за Y .
- Покажете **полиномиалност**: f се изчислява за $O(|x|^k)$ стъпки.

Верижност на редукциите

Важно свойство: ако $A \leq_p B$ и $B \leq_p C$, тогава $A \leq_p C$.

Това означава, че можем да изграждаме **вериги от редукции**:

$$\text{SAT} \leq_p \text{3-SAT} \leq_p \text{Vertex Cover} \leq_p \dots$$

Всяка задача в тази верига е NP-пълна (при условие, че е в NP).

Задача 1: $\text{SAT} \leq_p \text{3-SAT}$

Описание на задачите

SAT: Дадена е булева формула в КНФ ϕ . Съществува ли присвояване на променливите, което прави ϕ True?

3-SAT: Дадена е булева формула в КНФ, в която **всеки дизюнкт съдържа точно 3 литерала**. Съществува ли изпълняващо присвояване?

Защо 3-SAT е интересна?

SAT е очевидно обща (дизюнктите могат да имат произволна дължина). 3-SAT е *по-специална* — ограничена до дизюнкти с точно 3 литерала. Въпросът е: загубили ли сме мощност с това ограничение?

Отговорът е **НЕ** — 3-SAT е също толкова трудна като SAT (NP-пълна).

Конструкция на редукцията

Дадена е SAT формула ϕ в КНФ. Трансформираме всеки дизюнкт в еквивалентни 3-литерални дизюнкти.

Случай 1: Дизюнкт с 1 литерал $C = (L_1)$. Въвеждаме 2 нови помощни променливи q_1, q_2 :

$$C' = (L_1 \vee q_1 \vee q_2) \wedge (L_1 \vee q_1 \vee \neg q_2) \wedge (L_1 \vee \neg q_1 \vee q_2) \wedge (L_1 \vee \neg q_1 \vee \neg q_2)$$

Тези 4 дизюнкта са удовлетворими $\iff L_1 = \text{True}$ (само присвояването $L_1 = T$ удовлетворява всичките четири).

Случай 2: Дизюнкт с 2 литерала $C = (L_1 \vee L_2)$. Въвеждаме 1 нова помощна променлива q :

$$C' = (L_1 \vee L_2 \vee q) \wedge (L_1 \vee L_2 \vee \neg q)$$

C' е удовлетворима $\iff C$ е удовлетворима (ако $L_1 = L_2 = F$, нито $q = T$, нито $q = F$ спасяват и двата дизюнкта).

Случай 3: Дизюнкт с точно 3 литерала $C = (L_1 \vee L_2 \vee L_3)$. Оставяме без промяна: $C' = C$.

Случай 4: Дизюнкт с $k > 3$ литерала $C = (L_1 \vee L_2 \vee \dots \vee L_k)$. Въвеждаме $k - 3$ нови помощни променливи $q_1, q_2, \dots, q_{k-3}$ и заменяме с $k - 2$ нови дизюнкта:

$$C' = (L_1 \vee L_2 \vee q_1) \wedge (\neg q_1 \vee L_3 \vee q_2) \wedge (\neg q_2 \vee L_4 \vee q_3) \wedge \dots \wedge (\neg q_{k-3} \vee L_{k-1} \vee L_k)$$

Пример

Дизюнкт $C = (x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5)$ с 5 литерала. Въвеждаме q_1, q_2 :

$$C' = (x_1 \vee x_2 \vee q_1) \wedge (\neg q_1 \vee x_3 \vee q_2) \wedge (\neg q_2 \vee x_4 \vee x_5)$$

Проверка с $x_3 = T$: Задаваме $q_1 = T$ (удовлетворява дизюнкт 1 чрез q_1), $q_2 = F$. Тогава дизюнкт 2 е: $F \vee T \vee F = T$, а дизюнкт 3 е: $T \vee x_4 \vee x_5 = T$. Работи!

Коректност (Случай $k > 3$)

Посока ($\Rightarrow$): Ако C е удовлетворима, тогава C' е удовлетворима.

Нека $L_i = \text{True}$ за някое i . Задаваме:

- $q_j = \text{True}$ за всички $j < i - 1$ (удовлетворяват дизюнктите *преди* този с L_i).
- $q_j = \text{False}$ за всички $j \geq i - 1$ (удовлетворяват дизюнктите *след* този с L_i).
- Дизюнктът съдържащ L_i е директно удовлетворен от L_i .

Посока ($\Leftarrow$): Ако C' е удовлетворима, тогава C е удовлетворима.

Допускаме противното: всички $L_i = \text{False}$.

- Дизюнкт 1: $(L_1 \vee L_2 \vee q_1) = (F \vee F \vee q_1)$ — за да е True, трябва $q_1 = T$.
- Дизюнкт 2: $(\neg q_1 \vee L_3 \vee q_2) = (F \vee F \vee q_2)$ — трябва $q_2 = T$.
- По индукция: $q_{k-3} = T$.
- Последен дизюнкт: $(\neg q_{k-3} \vee L_{k-1} \vee L_k) = (F \vee F \vee F) = \text{False}$.

Противоречие! Значи поне един L_i е True.

Полиномиалност

- За дизюнкт с k литерала: добавяме $k - 3$ нови променливи и $k - 2$ нови дизюнкта.
- Общото нарастване е линейно спрямо размера на формулата.
- Трансформацията е $O(|\phi|)$.

Задача 2: $3\text{-SAT} \leq_p \text{Vertex Cover}$

Описание на задачите

3-SAT: Дадена е 3-CNF формула ϕ с n променливи и m дизюнкта. Изпълнима ли е?

Vertex Cover: Даден е граф $G = (V, E)$ и число k . Съществува ли множество $S \subseteq V$ с $|S| \leq k$, такова че за всяко ребро $(u, v) \in E$ — поне единият от u или v е в S ?

Интуиция

Ще “кодираме” присвояването на булевите променливи в структурата на граф. “Покриването” на ребрата ще съответства на “удовлетворяването” на дизюнктите.

Конструкция на графа G_ϕ

Стъпка 1 — Джаджи за променливи: За всяка променлива x_i създаваме **ребро** $(x_i, \neg x_i)$.

x_i ----- (не) x_i

Свойство: В коректно Vertex Cover трябва да изберем **точно един** от двата върха (поне един — за да покроем реброто; ако вземем и двата, “харчим” излишно). Избраният връх = **True**; неизбраният = **False**.

Стъпка 2 — Джаджи за дизюнкти: За всеки дизюнкт $C_j = (L_a \vee L_b \vee L_c)$ създаваме **триъгълник** от 3 нови върха $v_{j,1}, v_{j,2}, v_{j,3}$:

$v_{j,1}$
/ \
 $v_{j,3}$ ---- $v_{j,2}$

Свойство: За да покроем всички 3 ребра на триъгълника, трябва да изберем **поне 2** от 3-те върха.

Стъпка 3 — Свързващи ребра: Свързваме всеки връх от триъгълника с неговия литерален дубликат:

- $v_{j,1}$ свързан с L_a (литералният връх от джаджата за L_a).
- $v_{j,2}$ свързан с L_b .
- $v_{j,3}$ свързан с L_c .

Параметър: $k = n + 2m$.

Размер на графа

Компонент	Брой върхове	Брой ребра
Джаджи за променливи	$2n$	n
Джаджи за дизюнкти	$3m$	$3m$
Свързващи ребра	—	$3m$
Общо	$2n + 3m$	$n + 6m$

Конструкцията е $O(n + m)$ — полиномиална.

Коректност

Посока ($\Rightarrow$): Ако ϕ е удовлетворима, G_ϕ има Vertex Cover с размер $k = n + 2m$.

Нека дадено присвояване удовлетворява ϕ . Строим Vertex Cover S :

1. За всяка променлива x_i : добавяме **True** литерала към S (n върха).
2. За всеки дизюнкт C_j : поне един литерал е True $\rightarrow$ неговият връх от триъгълника е свързан с True литерален връх (вече в S , покрива реброто). Добавяме **другите два** върха от триъгълника към S ($2m$ върха).

Общо $|S| = n + 2m = k$.

Проверка: всяко ребро е покрито?

- Ребро в джаджа за променлива: покрито от избрания True литерален връх.
- Ребро в триъгълник: двата избрани върха покриват всичките 3 ребра на триъгълника.
- Свързващо ребро $(v_{j,i}, L)$: ако $L \in S$ (True) — покрито от L . Ако $L \notin S$ (False) — тогава $v_{j,i}$ е сред двата добавени върха от триъгълника, значи е в S .

Посока ($\Leftarrow$): Ако G_ϕ има Vertex Cover S с $|S| \leq n + 2m$, то ϕ е удовлетворима.

- За да покроем n ребра в джаджите за променливи: трябва **поне n** върха (по 1 от всяка двойка).
- За да покроем триъгълниците: трябва **поне $2m$** върха (по 2 от всеки триъгълник).
- Тъй като $|S| = n + 2m$: S съдържа **точно 1** от всяка двойка (дефинира присвояване) и **точно 2** от всеки триъгълник.

За всеки дизюнкт: точно 1 връх от триъгълника **не е** в S . Свързващото ребро между него и литералния му двойник трябва да е покрито $\rightarrow$ литералният връх **е** в $S \rightarrow$ литералът е True $\rightarrow$ дизюнкът е удовлетворен.

Задача 3: Subset Sum $\leq_p$ Partition

Описание на задачите

Subset Sum: Дадено е множество от цели положителни числа $A = \{a_1, \dots, a_n\}$ и число K . Съществува ли $I \subseteq A$, такова че $\sum_{x \in I} x = K$?

Partition: Дадено е множество A . Може ли да се раздели на две непресичащи се подмножества Y, Z , такива че $\sum_{y \in Y} y = \sum_{z \in Z} z$?

Интуиция

Partition е “симетрична” версия на Subset Sum — вместо да търсим подмножество с точна сума K , искаме два равни дяла. Ще “вградим” инстанция на Subset Sum в Partition чрез добавяне на един умен елемент.

Конструкция

Нека $S = \sum_{i=1}^n a_i$ (сумата на всички елементи). Добавяме нов елемент:

$$b = S - 2K$$

(Приемаме $S \geq 2K$; ако $S < 2K$, добавяме $b = 2K - S$ — симетричен случай.)

$$A' = A \cup \{b\}$$

Пример

$A = \{3, 1, 4, 2\}$, $K = 4$, $S = 10$.

$$b = 10 - 2 \cdot 4 = 2.$$

$A' = \{3, 1, 4, 2, 2\}$, обща сума = 12, всяка половина трябва да е 6.

Subset Sum решение: $\{1, 3\} = 4$. Така $I = \{1, 3\}$, $b = 2$.

$Y = \{1, 3, 2\} = 6$, $Z = \{4, 2\} = 6$. Работи!

Защо тоталната сума е $2S - 2K$?

$$\Sigma_{A'} = S + b = S + (S - 2K) = 2S - 2K$$

За валидно разбиване двете половини имат сума:

$$\frac{2S - 2K}{2} = S - K$$

Коректност

Посока ($\Rightarrow$): Ако $\exists I \subseteq A$ с $\sum I = K$, тогава A' може да се раздели.

Дефинираме:

- $Y = I \cup \{b\}$: сума = $K + (S - 2K) = S - K$.
- $Z = A \setminus I$: сума = $S - K$.

Посока ($\Leftarrow$): Ако A' може да се раздели, тогава $\exists I \subseteq A$ с $\sum I = K$.

Нека Y, Z е разбиването, всяко със сума $S - K$. Елементът b е в точно едно от тях, без ограничение на общността $b \in Y$. Дефинираме $I = Y \setminus \{b\}$.

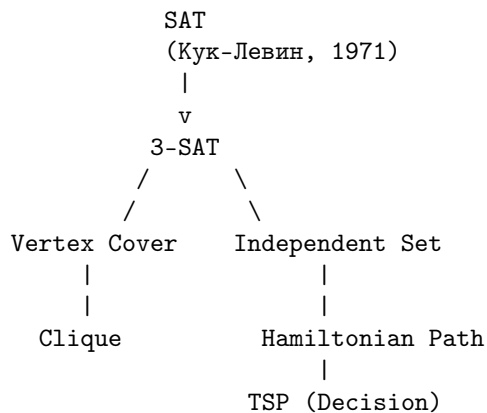
$$\sum_{x \in I} x = \sum Y - b = (S - K) - (S - 2K) = K$$

Полиномиалност

- Изчисляване на S : $O(n)$.
- Добавяне на b : $O(1)$.
- Общо: $O(n)$.

Обобщение

Верига от редукции:



Subset Sum --> Partition --> Knapsack

Ключови принципи:

Принцип	Описание
$P \subseteq NP$	Всяко P е в NP (можем да проверяваме като решаваме)
$P = NP?$	Великият отворен въпрос; вероятно $P \neq NP$
NP -пълнота = $NP \cap NP$ -hard	Най-трудните задачи в NP
Редукция $A \leq_p B$	A не е по-трудна от B ; B е поне толкова трудна
Посока	Известна трудна $\leq_p$ Нова задача (за NP -пълнота)
Верижност	$A \leq_p B$ и $B \leq_p C \Rightarrow A \leq_p C$